

2.3 改进分治算法的途径

- 平面点对问题算法描述: $\text{MinDistance}(P, X, Y)$

输入: n 个点的集合 P , X 和 Y 分别为横、纵坐标数组

输出: 最近的两个点及距离

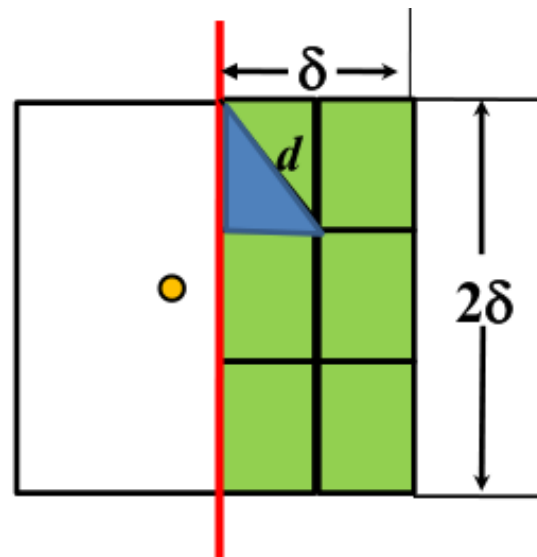
1. 如果 P 中点数小于等于3, 则直接计算其中的最小距离
2. 排序 X, Y
3. 做垂直线 l 将 P 划分为 P_L 和 P_R , P_L 的点在 l 左边, P_R 的点在 l 右边
4. $\delta_L = \text{MinDistance}(P_L, X_L, Y_L)$; P_L 中的最小距离
5. $\delta_R = \text{MinDistance}(P_R, X_R, Y_R)$; P_R 中的最小距离
6. $\delta = \min(\delta_L, \delta_R)$
7. 对于在 l 线左边距离 δ 内每个点, 检查右边是否有与之距离小于 δ 的点, 如果存在则将 δ 修改为新值



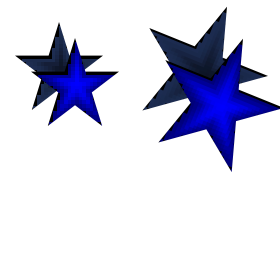
2.3 改进分治算法的途径

- 跨边界处理

$$\begin{aligned}d &= \sqrt{(\delta/2)^2 + (2\delta/3)^2} \\&= \sqrt{\delta^2/4 + 4\delta^2/9} \\&= \sqrt{25\delta^2/36} = 5\delta/6\end{aligned}$$



右边每个小方格至多1个点，每个点至多比较对面的6个点，检查1个点是常数时间， $O(n)$ 个点需要 $O(n)$ 。



2.3 改进分治算法的途径

- 算法分析

步1 递归边界处理: $O(1)$

步2 排序: $O(n\log n)$

步3 划分: $O(1)$

步4-5子问题: $2T(n/2)$

步6确定 δ : $O(1)$

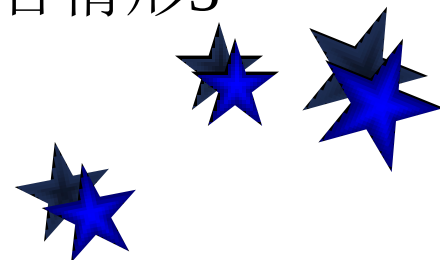
步7检查跨边界点对: $O(n)$

$$T(n) = 2T(n/2) + O(n\log n)$$

$$T(n) = O(1), n \leq 3$$

不能使用主方法求解，不符合情形3

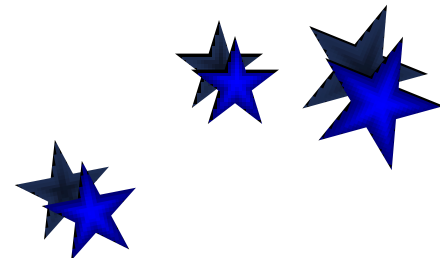
递归树求解 $T(n) = O(n\log^2 n)$



2.3 改进分治算法的途径

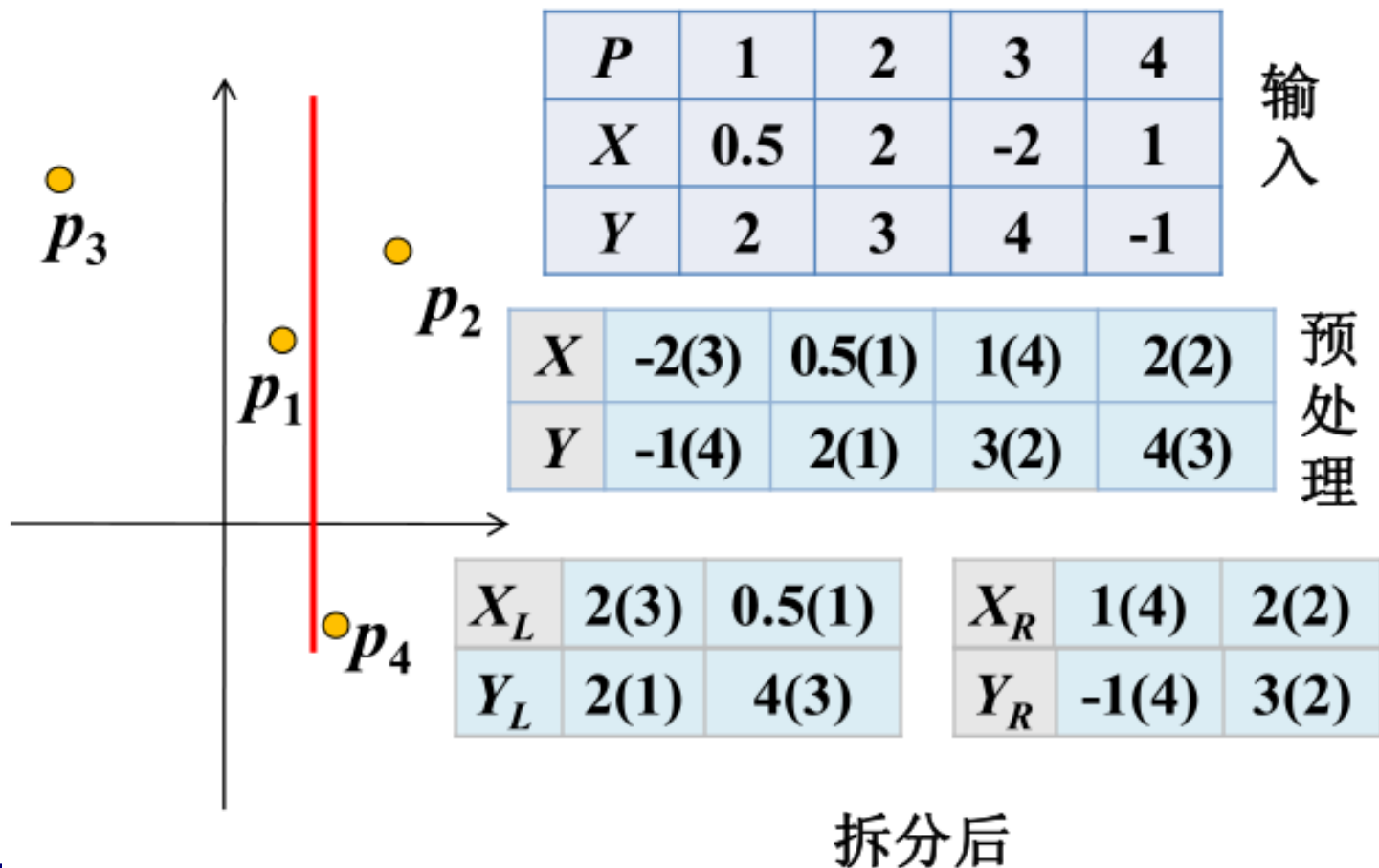
- 改进方法：增加预处理
 - (1) 原算法效率低的原因：在每次划分时对子问题数组重新排序。
 - (2) 改进策略
 - * 在递归前对 $\mathbf{X}, \mathbf{Y}$ 排序，作为预处理
 - * 划分时对排序的数组 $\mathbf{X}, \mathbf{Y}$ 进行拆分，得到针对子问题 $\mathbf{P}_L$ 的数组 $\mathbf{X}_L, \mathbf{Y}_L$ 针对子问题 $\mathbf{P}_R$ 的数组 $\mathbf{X}_R, \mathbf{Y}_R$

原问题规模为 n ，拆分的时间为 $O(n)$



2.3 改进分治算法的途径

(3) 实例：递归中的拆分



2.3 改进分治算法的途径

(4) 改进算法时间复杂度

$W(n)$ 为算法时间复杂度

递归过程: $T(n)$, 预处理: $O(n\log n)$

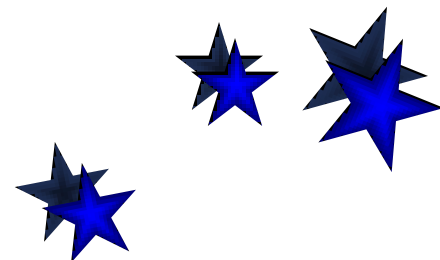
$$W(n) = T(n) + O(n\log n)$$

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(1) \quad n \leq 3$$

解得 $T(n) = O(n\log n)$

于是 $W(n) = O(n\log n)$



2.3 改进分治算法的途径

■ 小结

- 依据

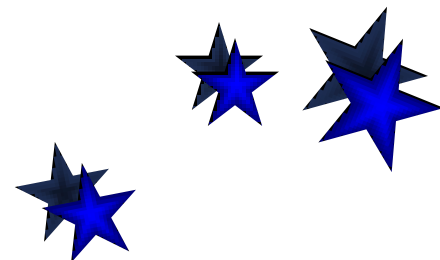
$$W(n) = aW(n/b) + f(n)$$

- 提高算法效率的方法:

- 减少子问题个数 a :

$$W(n) = O(n^{\log_b a})$$

- 增加预处理, 减少 $f(n)$



2.4 典型实例

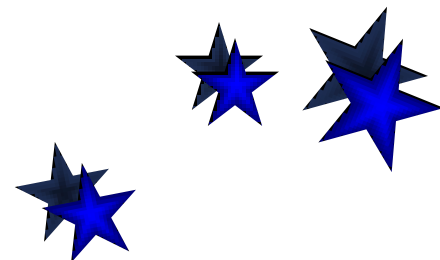
■ 选择问题

- 选最大
- 选最大最小
- 选第二大
- 选第 k 小

■ 多项式乘积

- 多项式乘积的应用——卷积计算
- 卷积计算的应用——信号的平滑处理
- 快速傅立叶变换FFT算法

■ 计算几何：计算平面点集的凸包（作业）



2.4 典型实例

■ 选择问题

输入：集合 L (含 n 个不等的实数)

输出： L 中第 i 小元素

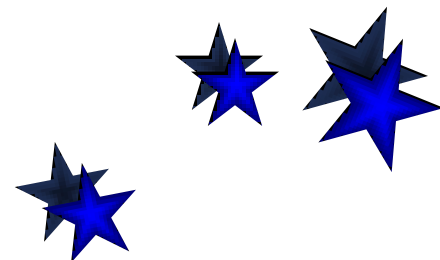
$i=1$, 称为最小元素

$i=n$, 称为最大元素

位置处在中间的元素, 称为中位元素

n 为奇数, 中位数唯一, $i = (n+1)/2$

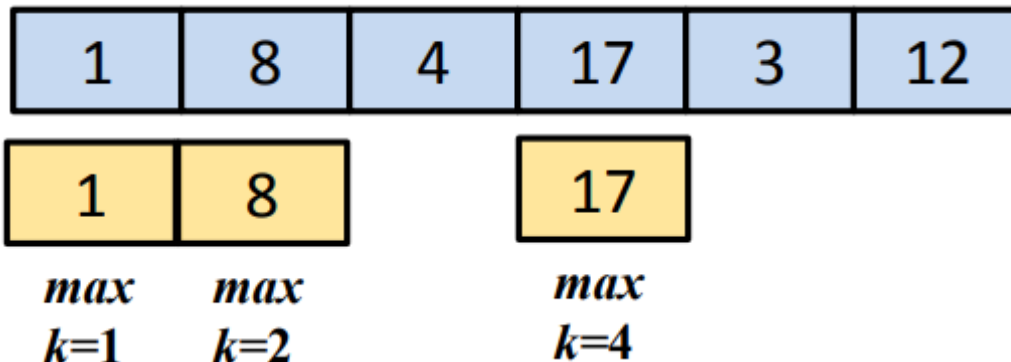
n 为偶数, 可指定 $i = n/2+1$



2.4 典型实例

□ 选最大

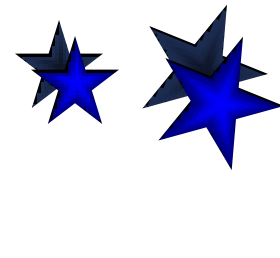
- 实现：顺序比较



输出： $\text{max} = 17$, $k=4$

算法最坏情况下的时间 $W(n)=n-1$

最好效率



2.4 典型实例

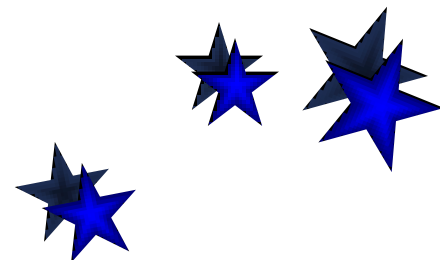
- 算法实现

算法 Findmax

输入： n 个数的数组 L

输出： max, k

1. $max \leftarrow L[1]$
2. for $i \leftarrow 2$ to n do
3. if $max < L[i]$
4. then $max \leftarrow L[i]$
5. $k \leftarrow i$
6. return max, k



2.4 典型实例

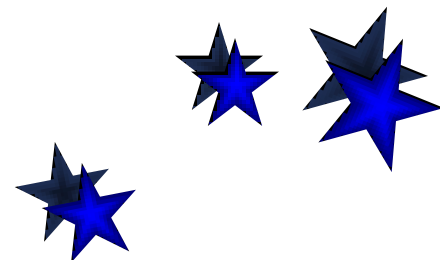
□ 选最大最小

- 实现算法1：顺序比较

1. 顺序比较，先选最大 *max*
2. 顺序比较，在剩余数组中选最小 *min*，类似于选最大算法，但比较时保留较小的数

时间复杂性：

$$W(n) = n-1 + n-2 = 2n-3$$



2.4 典型实例

- 实现算法2：分组算法

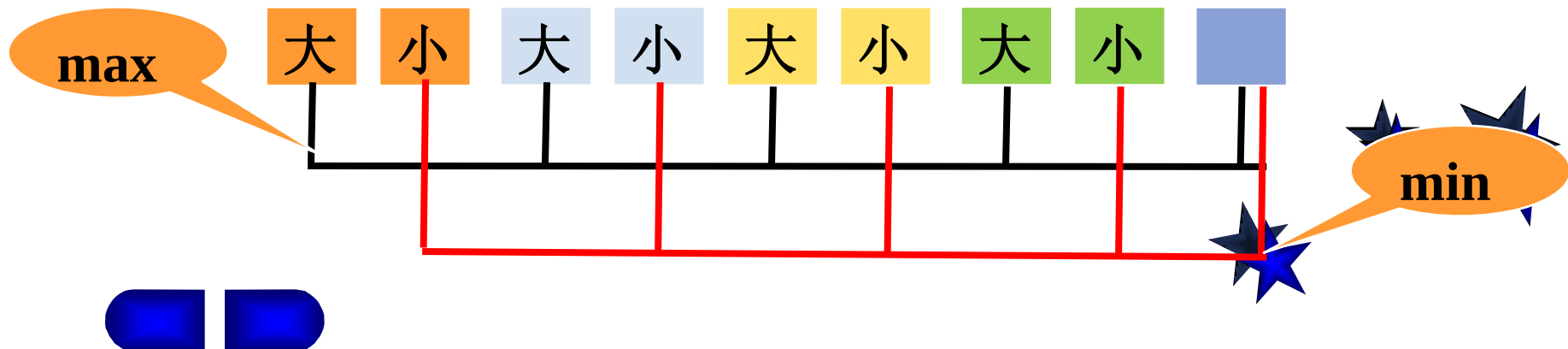
(1) 将所有的数两两分到一组，剩余的一个元素轮空



(2) 比较每组的两个数，大数放在前，小数放在后



(3) 在所有大及轮空中选最大，在所有小及轮空中选小



2.4 典型实例

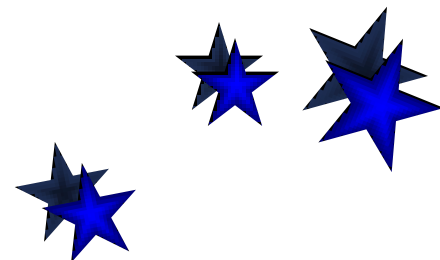
- 算法实现

算法 **FindMaxMin**

输入： n 个数的数组 L

输出： max , min

1. 将 n 个元素两两一组分成 $\lfloor n/2 \rfloor$ 组
2. 每组比较，得到 $\lfloor n/2 \rfloor$ 个较小和 $\lfloor n/2 \rfloor$ 个较大
3. 在 $\lceil n/2 \rceil$ 个较大（含轮空元素）中找最大 max
4. 在 $\lceil n/2 \rceil$ 个较小（含轮空元素）中找最小 min



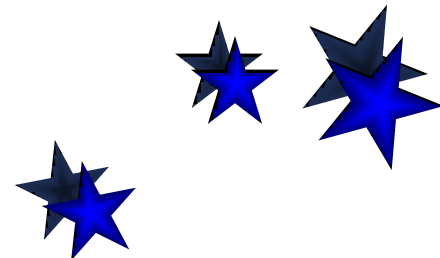
2.4 典型实例

- 最坏情况时间复杂度

行2 的组内比较: $\lfloor n/2 \rfloor$ 次

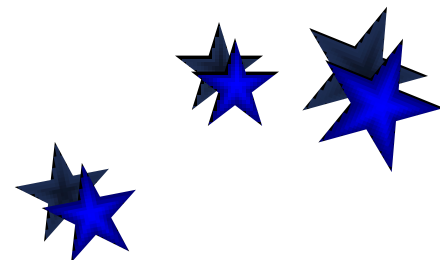
行3--4 求 *max* 和 *min* 比较:
至多 $2\lceil n/2 \rceil - 2$ 次

$$\begin{aligned} W(n) &= \lfloor n/2 \rfloor + 2\lceil n/2 \rceil - 2 \\ &= n + \lceil n/2 \rceil - 2 \\ &= \lceil 3n/2 \rceil - 2 \end{aligned}$$



2.4 典型实例

- 实现算法3：分治算法
 1. 将数组 L 从中间划分为两个子数组 L_1 和 L_2
 2. 递归地在 L_1 中求最大 max_1 和 min_1
 3. 递归地在 L_2 中求最大 max_2 和 min_2
 4. $max \leftarrow \max\{max_1, max_2\}$
 5. $min \leftarrow \min\{min_1, min_2\}$



2.4 典型实例

- 最坏情况时间复杂度

假设 $n = 2^k$,

$$W(n) = 2W(n/2) + 2$$

$$W(2) = 1$$

解 $W(2^k) = 2W(2^{k-1}) + 2$

$$= 2[2W(2^{k-2}) + 2] + 2$$

$$= 2^2 W(2^{k-2}) + 2^2 + 2 = \dots$$

$$= 2^{k-1} + 2^{k-1} + \dots + 2^2 + 2$$

$$= 3 \cdot 2^{k-1} - 2 = 3n/2 - 2$$

最好效率



2.4 典型实例

□ 选第二大

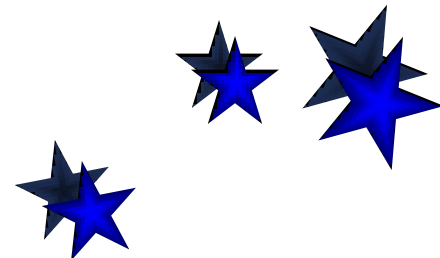
输入： n 个数的数组 L

输出： 第二大的数 $second$

- 实现算法1： 顺序比较
 1. 顺序比较找到最大 max
 2. 从剩下 $n-1$ 个数中找最大，就是第二大 $second$

时间复杂度：

$$W(n) = n - 1 + n - 2 = 2n - 3$$



2.4 典型实例

- 提高效率的途径

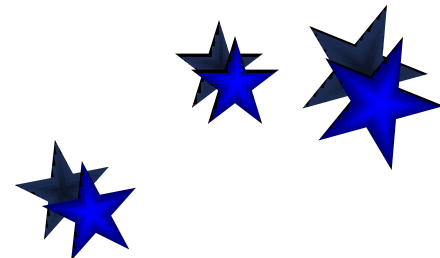
(1) 成为第二大数的条件：仅在与最大数的比较中被淘汰。

(2) 要确定第二大数，必须知道最大数。

(3) 在确定最大数的过程中记录下被最大数直接淘汰的元素。

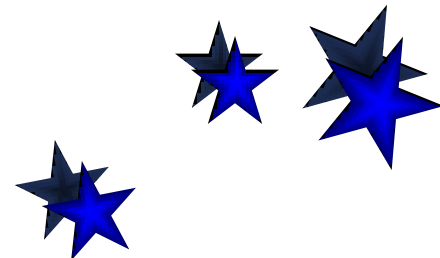
(4) 在被最大数直接淘汰的数中找到的最大数就是第二大数。

设计思想： 用空间换时间。



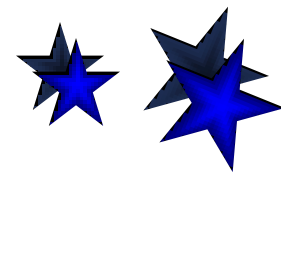
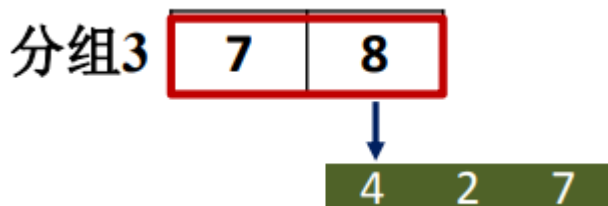
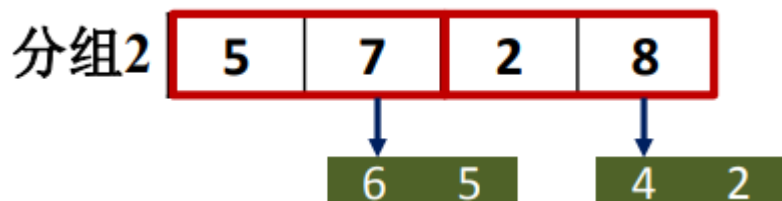
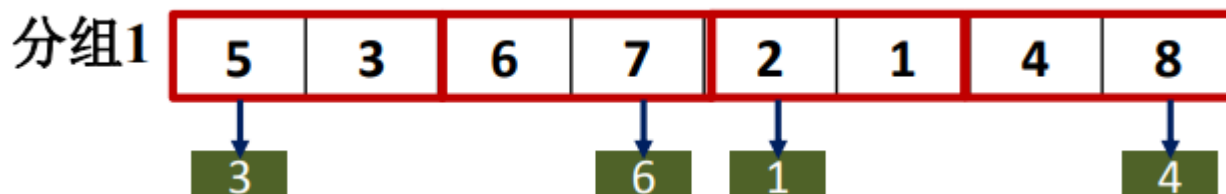
2.4 典型实例

- 实现算法2：锦标赛算法
 1. 两两分组比较，大者进入下一轮，直到剩下 1 个元素 *max* 为止
 2. 在每次比较中淘汰较小元素，将被淘汰元素记录在淘汰它的元素的链表上
 3. 检查 *max* 的链表，从中找到最大元，即 *second*



2.4 典型实例

实例演示



2.4 典型实例

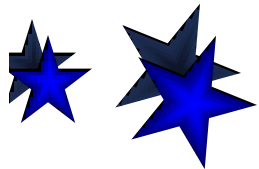
算法 FindSecond

输入: n 个数的数组 L , 输出: $second$

1. $k \leftarrow n$ // 参与淘汰的元素数
2. 将 k 个元素两两1组, 分成 $\lfloor k/2 \rfloor$ 组
3. 每组的2个数比较, 找到较大数
4. 将被淘汰数记入较大数的链表
5. if k 为奇数 then $k \leftarrow \lfloor k/2 \rfloor + 1$
6. else $k \leftarrow \lfloor k/2 \rfloor$
7. if $k > 1$ then goto 2
8. $max \leftarrow$ 最大数
9. $second \leftarrow max$ 的链表中的最大

一轮淘汰

继续分组淘汰



2.4 典型实例

- 时间复杂度分析

命题1 设参与比较的有 t 个元素，经过 i 轮淘汰后元素数至多为 $\lceil t/2^i \rceil$.

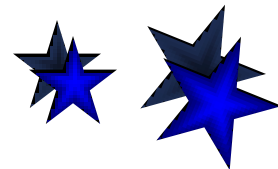
证 对 i 归纳. $i=1$, 分 $\lfloor t/2 \rfloor$ 组，淘汰 $\lfloor t/2 \rfloor$ 个元素，进入下一轮元素数是

$$t - \lfloor t/2 \rfloor = \lceil t/2 \rceil$$

假设 i 轮分组淘汰后元素数至多为 $\lceil t/2^i \rceil$,

那么 $i+1$ 轮分组淘汰后元素数为

$$\lceil \lceil t/2^i \rceil / 2 \rceil = \lceil t/2^{i+1} \rceil$$



2.4 典型实例

命题2 *max* 在第一阶段分组比较中总计进行了 $\lceil \log n \rceil$ 次比较.

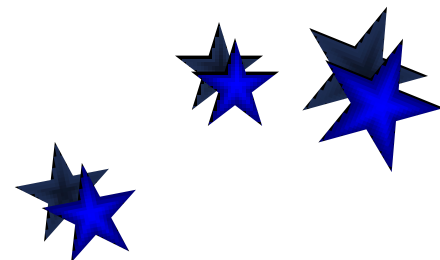
证 假设到产生 *max* 时总计进行 k 轮淘汰, 根据命题 1 有 $\lceil n/2^k \rceil = 1$.

若 $n=2^d$, 那么有

$$k = d = \log n = \lceil \log n \rceil$$

若 $2^d < n < 2^{d+1}$, 那么

$$k = d + 1 = \lceil \log n \rceil$$



2.4 典型实例

第一阶段元素数: n

比较次数: $n-1$

淘汰了 $n-1$ 个元素

锦标赛阶段，找最大值

第二阶段: 元素数 $\lceil \log n \rceil$

比较次数: $\lceil \log n \rceil - 1$

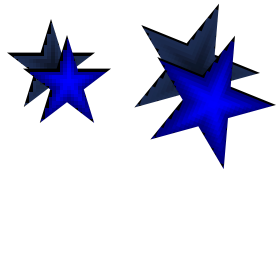
淘汰元素数为 $\lceil \log n \rceil - 1$

从max的淘汰链表中找出最大值

时间复杂度是

$$\begin{aligned} W(n) &= n - 1 + \lceil \log n \rceil - 1 \\ &= n + \lceil \log n \rceil - 2. \end{aligned}$$

空间换时间



2.4 典型实例

□ 一般性选择问题——选第 k 小

问题：选第 k 小。

输入：数组 S , S 的长度 n , 正整数 k ,

$1 \leq k \leq n$.

输出：第 k 小的数

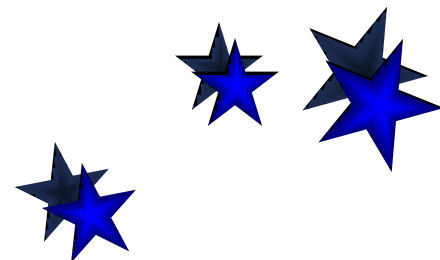
实例 1

$S = \{ 3, 4, 8, 2, 5, 9, 18 \}$, $k = 4$, 解: 5

实例 2

统计数据的集合 S , $|S|=n$,

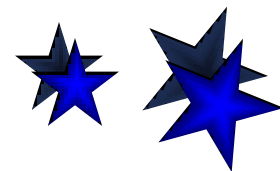
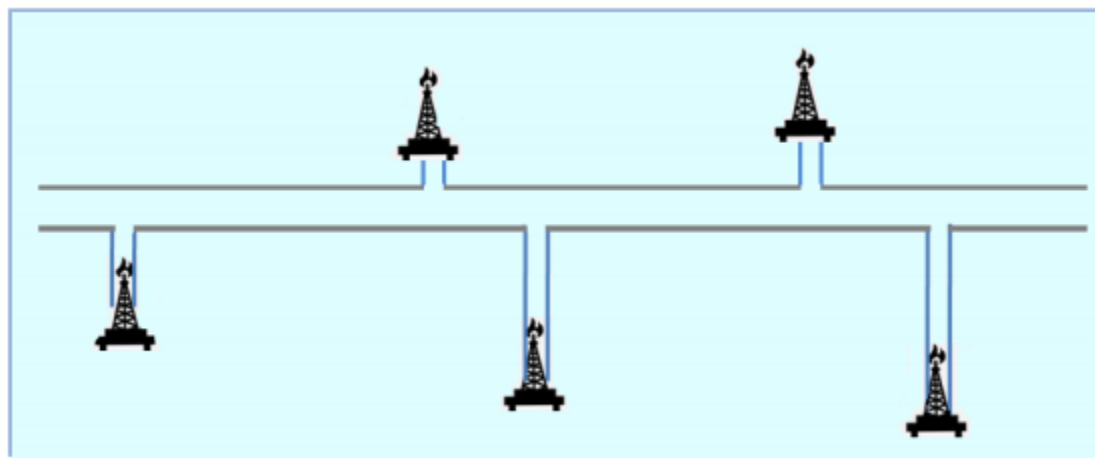
选中位数, $k = \lceil n/2 \rceil$



2.4 典型实例

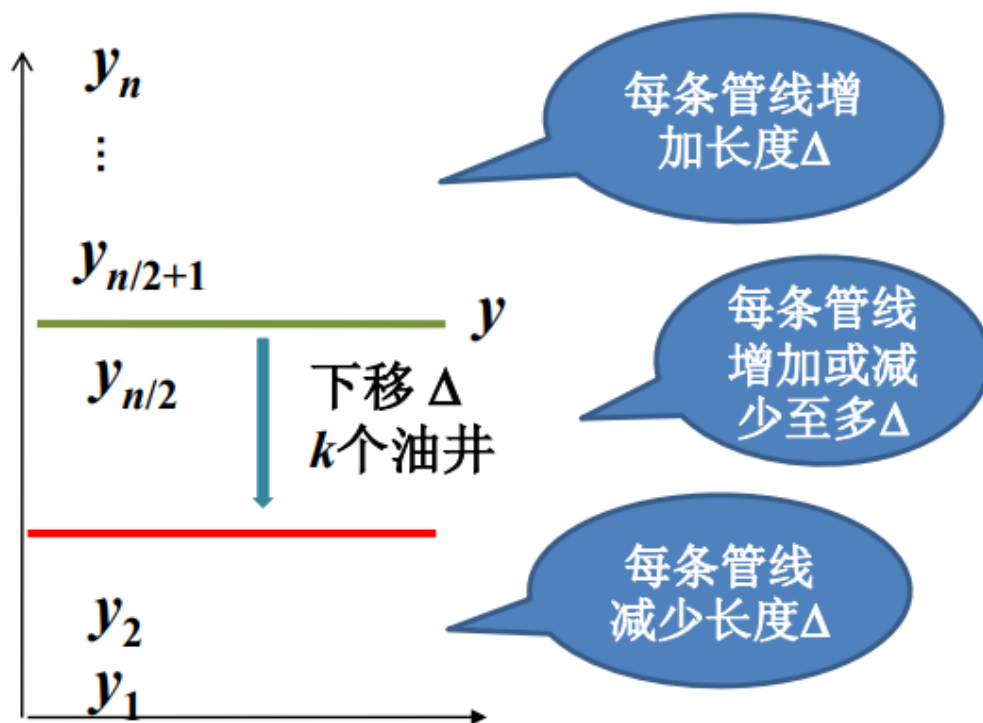
- 中位数应用举例——管道位置选取

问题：某区域有 n 口油井，需要修建输油管道. 根据设计要求，水平方向有一条主管道，每口油井修一条垂直方向的支管道通向主管道. 如何选择主管道的位置，以使得支管道长度的总和最小？



2.4 典型实例

问题的最优解： $\mathbf{Y}$ 坐标的中位数



- 原 y 之上的油井管道，其长度都增加了 Δ
- 原 y 之下的油井管道
 - (1) 新位置之下的均减少 Δ
 - (2) y 与新位置之间的或者增加、或者减少至多 Δ
- 移动到新位置后，长度增加的油井数多于长度减少的油井数。

下移后支管线总长度增加



2.4 典型实例

实现算法

(1) 算法一

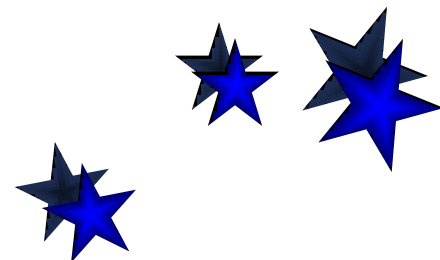
调用 k 次选最小算法

时间复杂度为 $O(kn)$

(2) 算法二

先排序，然后输出第 k 小的数

时间复杂度为 $O(n \log n)$



2.4 典型实例

(3) 算法三：分治策略

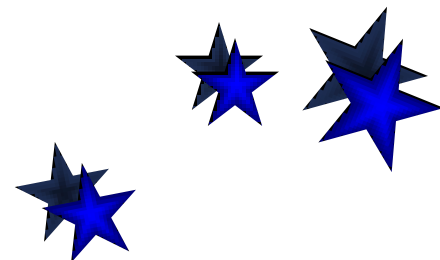
假设元素彼此不等，设计思想：

1. 用某个元素 m^* 作为标准将 S 划分成 S_1 与 S_2 ，其中 S_1 的元素小于 m^* ， S_2 的元素大于等于 m^* 。



2. 如果 $k \leq |S_1|$ ，则在 S_1 中找第 k 小。
如果 $k = |S_1| + 1$ ，则 m^* 是第 k 小。
如果 $k > |S_1| + 1$ ，则在 S_2 中找第 $k - |S_1| - 1$ 小。

算法效率取决于子问题规模



2.4 典型实例

? 如何通过 m^* 控制子问题规模? 实例: $n=15, k=6$

(1) 5个数一组，共划分成 $n/5$ 组

8	2	3	5	7	6	11	14	1	9	13	10	4	12	15
---	---	---	---	---	---	----	----	---	---	----	----	---	----	----

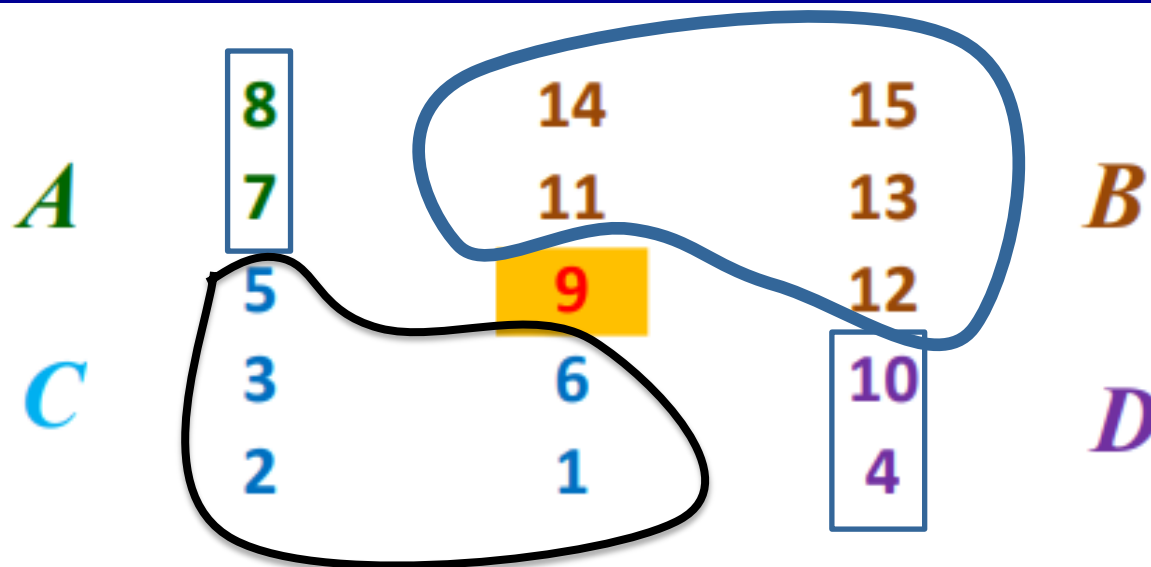
(2) 每组的5个数排序（例如：从大到小），则每组的中位数构成一维数组 M ， M 中共 $n/5$ 个数。

(3) 找到 M 的中位数，该数即为 m^* 。

	8	14	15
	7	11	13
M	5	9	12
	3	6	10
	2	1	4

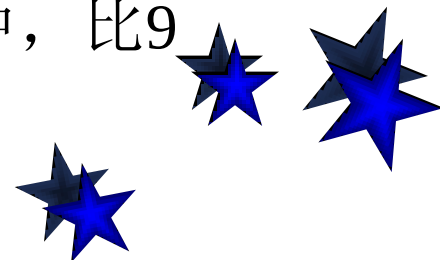
$m^* = 9$ ★ ★

2.4 典型实例

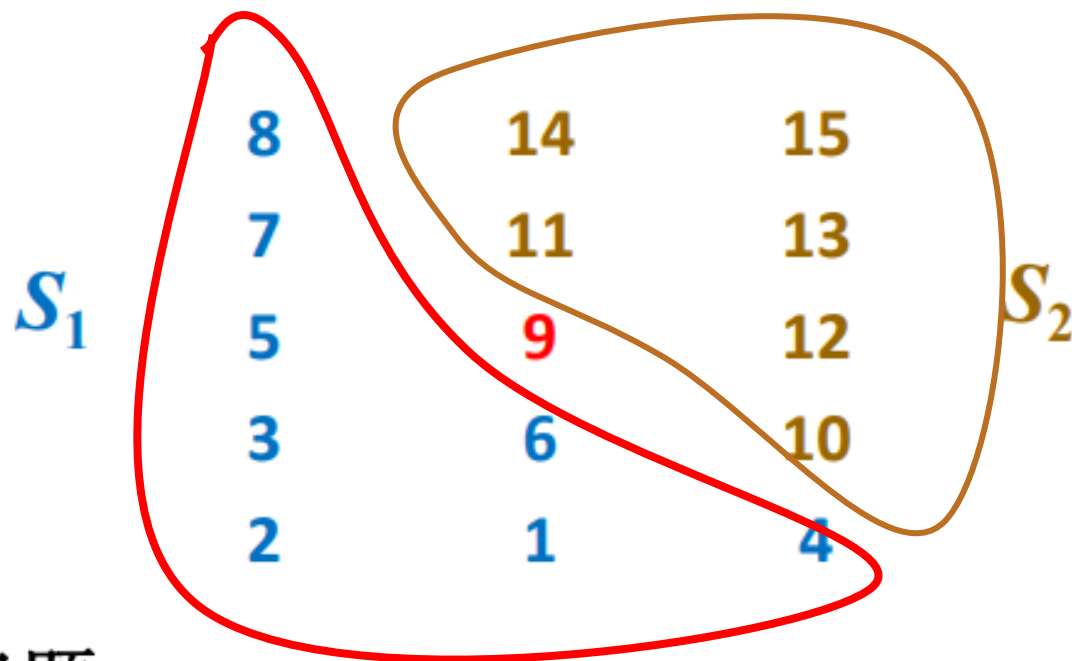


$A(8, 7)$, $D(10, 4)$ 需要与 $m^*(9)$ 比较

归约：将 A 、 D 区域中比 9 小的放入 C (S_1) 中，比 9 大的放入 B (S_2) 中，



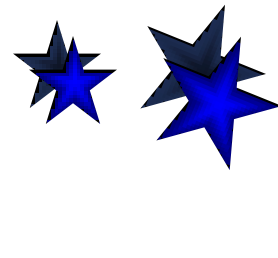
2.4 典型实例



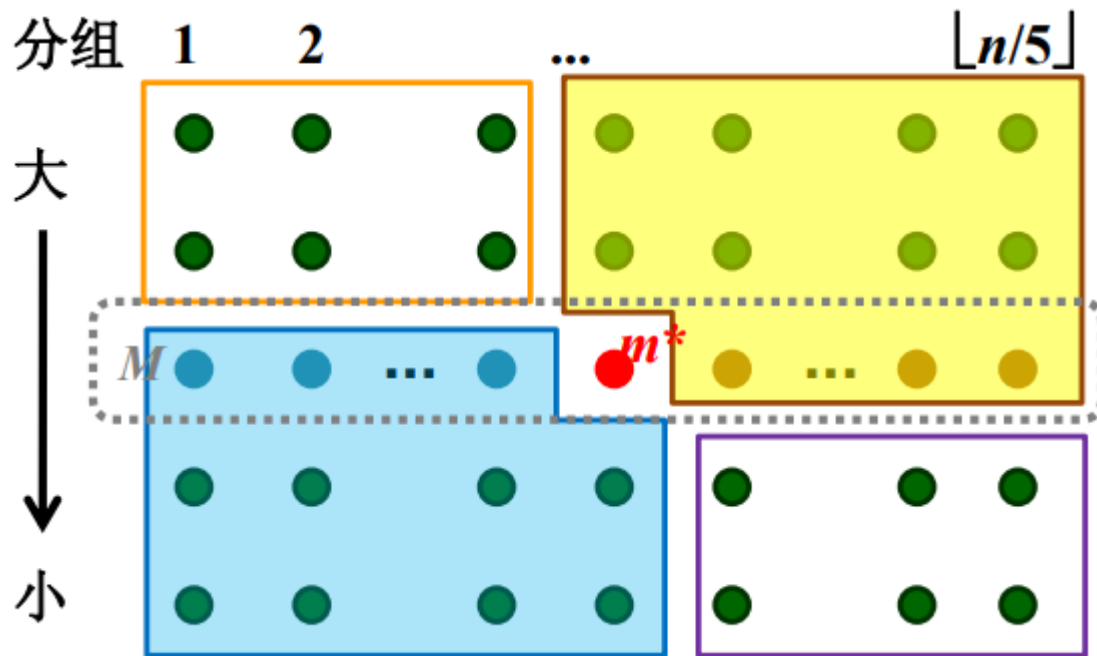
子问题

8 7 5 3 2 6 1 4

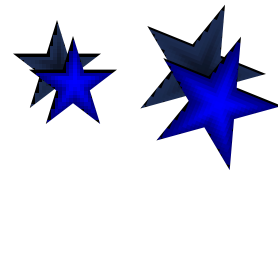
子问题规模 = 8, $k=6$



2.4 典型实例

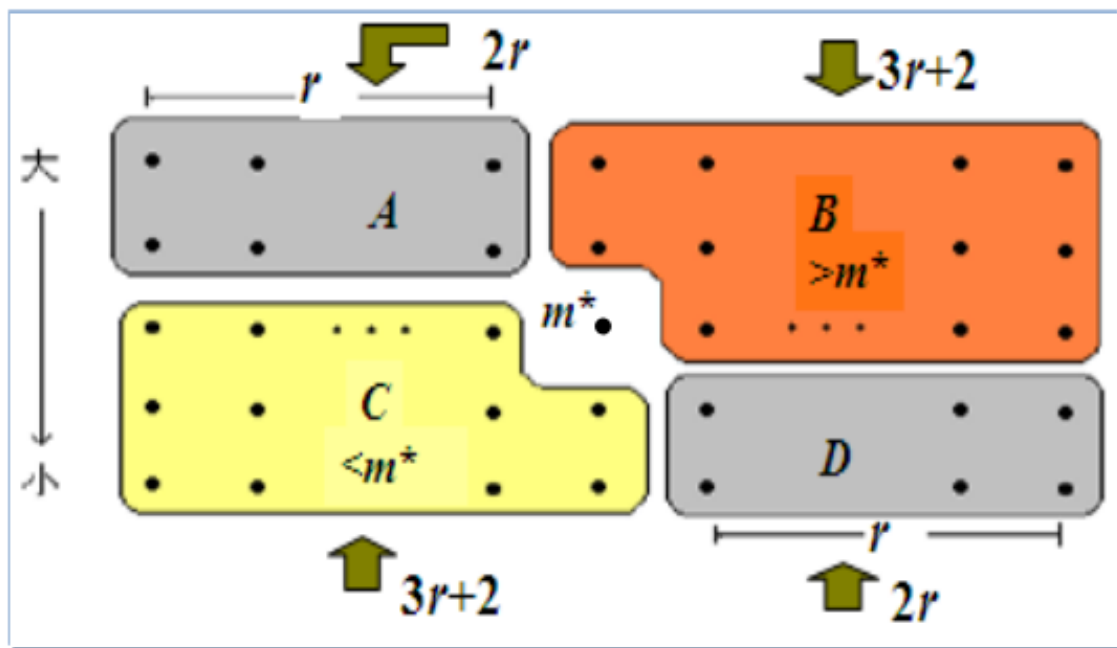


A: 数需要与 m^* 比大小, **B:** 数大于 m^*
C: 数小于 m^* , **D:** 数需要与 m^* 比大小

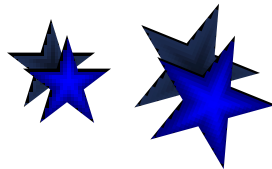


2.4 典型实例

■ 选择问题的算法分析



$n = 5(2r + 1), |A| = |D| = 2r$
子问题规模至多: $2r + 2r + 3r + 2 = 7r + 2$



2.4 典型实例

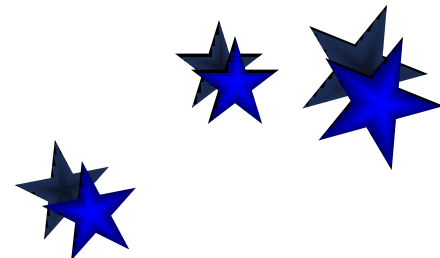
- 子问题规模估计

不妨设 $n = 5(2r + 1)$, $|A| = |D| = 2r$,

$$r = \frac{n/5 - 1}{2} = \frac{n}{10} - \frac{1}{2}$$

划分后子问题规模至多为

$$\begin{aligned} \underline{7r + 2} &= 7\left(\frac{n}{10} - \frac{1}{2}\right) + 2 \\ &= \frac{7n}{10} - \frac{3}{2} < \frac{7n}{10} \end{aligned}$$



2.4 典型实例

- 时间复杂度递推方程

算法工作量 $W(n)$

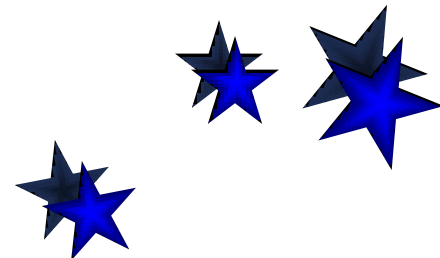
行2: $O(n)$ //每5个数找中位数,构成 M

行3: $W(n/5)$ // M 中找中位数 m^*

行4: $O(n)$ // 用 m^* 划分集合 S

行8-9: $W(7n/10)$ //递归

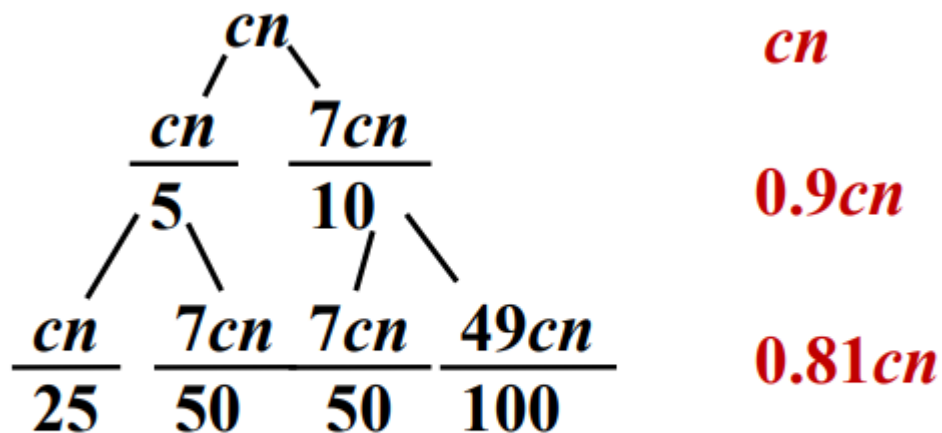
$$W(n) \leq W(n/5) + W(7n/10) + O(n)$$



2.4 典型实例

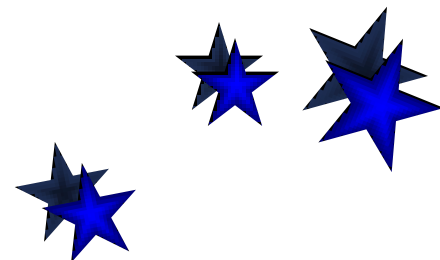
- 时间复杂度递推方程

$$W(n) = W(n/5) + W(7n/10) + cn$$



.....

$$W(n) \leq cn (1 + 0.9 + 0.9^2 + \dots) = O(n)$$

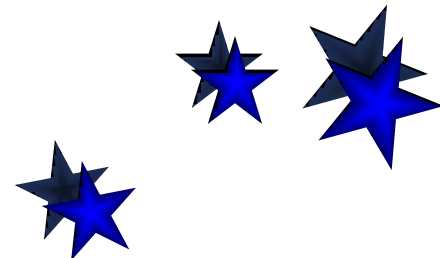


2.4 典型实例

- 讨论：分组时为什么5个元素一组？3个一组或7个一组行不行？

分析：递归调用

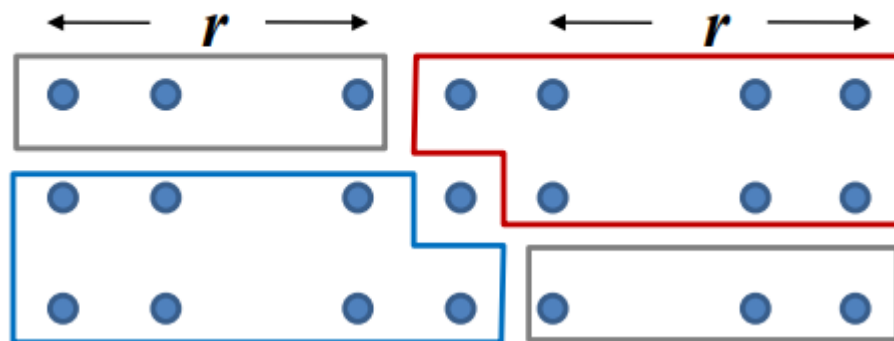
1. 求 m^* 的工作量与 $|M| = n/t$ 相关， t 为每组元素数。 t 大， $|M|$ 小
2. 归约后子问题大小与分组元素数 t 有关。 t 大，子问题规模大



2.4 典型实例

- 3分组时的子问题规模

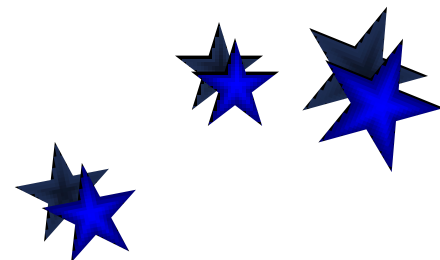
假设 $t=3$, 3个一组:



$$n = 3(2r + 1)$$

$$r = (n/3 - 1)/2 = n/6 - 1/2$$

子问题规模最多为 $4r+1 = 4n/6 - 1$



2.4 典型实例

- 3分组时的时间复杂度

算法的时间复杂度满足方程

$$W(n) = W(n/3) + W(4n/6) + cn$$

由递归树得 $W(n) = \Theta(n \log n)$

关键：

$|M|$ 与归约后子问题规模之和小于 n ，
递归树每行的工作量构成公比小于 1
的等比级数， 算法复杂度才是 $O(n)$.

